From WAV to Graphics to MIDI[1]

A.J. Allinger
Kruger Optical
Oct. 2021

Abstract: A method of transcription from audio to symbolic musical format is proposed as a process requiring two automated transformations:
1. Convert audio to image.
(Edit the image.)
2. Convert image to music.
A high-resolution spectrogram is made from a wave audio file. The constant-Q transform and the frequency reassignment algorithm are used for consistent high resolution. The user edits the image to erase overtones and echos. The edited image is then converted to MIDI. Dynamic programming and a nonlinear smoother remove noise and flutter. Greedy agglomeration merges frames into notes.

Table of Contents

- Introduction
- Part 1: Building a Better Spectrogram
    - Constant-Q Transform
    - Window Functions
    - Frequency Reassignment
    - Lanczos Interpolation
- Interlude: Editing the Image
- Part 2: From Melody Line to MIDI
    - Undoing Lanczos Interpolation with Root-finding
    - Separating Sound from Silence with Dynamic Programming
    - Trimming Outliers with a LULU Smoother
    - Agglomerating the Notes
    - The Formula for Expression
- Build Instructions
- Using the Code
- Release History
- References

1 This article originally appeared on codeproject.com as From WAV to Graphics to MIDI- CodeProject

Introduction

To transcribe recorded music to score notation is a difficult problem. Notes are hard to pick out from among the overtones and echos. On the other hand, converting solo audio to MIDI is fairly straightforward.

One possible method is to convert the audio to an image by taking the spectrogram. The image is then hand-edited to erase overtones and echos, leaving the most subtle step for the human brain. The solo line of the edited image may be converted to MIDI. The `SPECTR` and `SCRIBE` programs are intended to serve as a MIDI voice controller, comparable to a keyboard or a breath controller.

Program `SPECTR` produces a high-resolution spectrogram in the alto range, 32 notes in all, from F below treble clef to C above treble clef.

Next, the user edits the image to remove the overtones. Because this is a standard image file, any editing operation which can be done on an image may be done at this step.

Program `SCRIBE` converts the edited image to a standard MIDI file. The output of `SCRIBE` is made long by many changes in expression and pitch bend, at 240 frames per second. The shortcoming of the program is that it only transcribes solos, and it is a lot of work to hand-edit the images.

This is a command-prompt program. It is free and in the public domain.

Some subroutines are of interest in their own right. Subroutine `FRAT` implements the Frequency Reassignment transform, which offers greater precision than the Fourier transform. `GLOM` merges elements of a sequence into segments of similar value. `SMOOTH` is a nonlinear smoother for removing outliers from a data sequence.

Also included in the distribution is some excellent legacy code:

- `SOLV` and `ZERORA` are based on ZEROIN by L F Shampine and H A Watts for solving an equation.
- `ISHELL` by Alfred H. Morris, Jr. sorts an integer array.
- `RORDER` by Robert Renka applies a permutation to a floating-point array.

Part 1: Building a Better Spectrogram

A well-known graphical representation of a sound is the spectrogram. In this view, time is represented in the horizontal direction and frequency is represented in the vertical direction. Each vertical line in the image is a Fourier transform of a brief portion of the sound. Each pixel within that line is a coefficient of the Fourier transform.

For more information on the Fourier transform, please refer to the CodeProject articles:

- Discrete Fourier Transform for Frequency Analysis by pi19404, and
- Fourier Transform in Digital Signal Processing by Jakub Szymanowski

The Constant-Q Transform

The discrete Fourier transform takes a sequence and returns an array of coefficients (see Appendix). The original signal is represented as a sum of sine and cosine waves at frequencies $f_o,\ 2f_o, \cdots,\ if_o, \cdots,\ \frac{N}{2}f_o$ where f_o is the fundamental frequency, that is, the sampling rate divided by the number of samples included in the transform. The design equation for setting up a Fourier transform is:

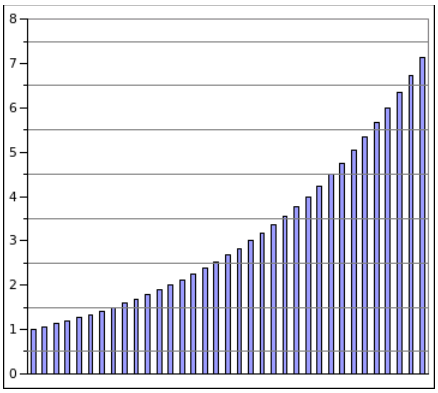
MR = NF

where:

- M is an integer
- R is the sampling rate in cycles per second
- N is the number of samples in the transform
- F is the frequency to be measured

The limitation of Fourier analysis is that in general, there is a tradeoff between time resolution and frequency resolution. To make the frequencies more closely spaced, f_o must be made smaller. For a given sample rate, this requires N be made larger. The more precisely you seek the frequency of an event, the less precisely you can determine when it happened.

By its nature, the Fourier transform finds amplitudes at a set of equally spaced frequencies. For the purposes of musical analysis, this is not optimal. The tones of the musical scale are spaced according to a geometric progression. Specifically, each tone has a frequency greater than the previous by a factor of $\sqrt[12]{2}$.



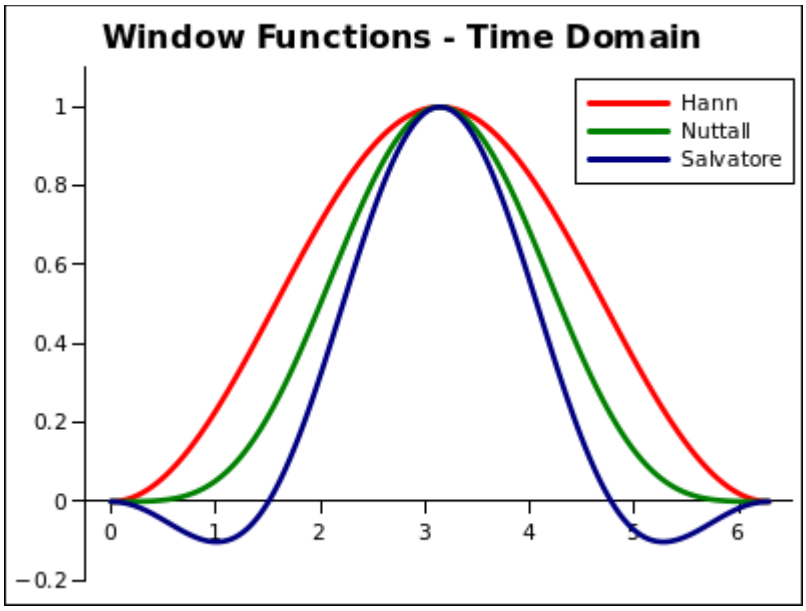
The vertical bars in the graph represent the exponential growth of frequencies of musical tones. The horizontal lines represent the evenly spaced Fourier transform bins. A transform size adequate to separate notes of high frequency will not separate notes of low frequency. A transform size adequate to separate notes of low frequency has excess resolution for notes of

high frequency. A large transform size must be chosen. This comes at the expense of diminished time resolution and increased computation.

A straightforward solution [Brown, 1991] is to take transforms of different sizes to observe the different frequencies. This is the Constant-Q transform, which is not a new mathematical technique, but a targeted use of the Fourier transform. It allows the frequency-time resolution tradeoff to be chosen in an optimal way across the spectrum. In program SPECTR, M is chosen as 17, because the ratios 16:17 and 17:18 are approximately equal to the spacing between tones of the musical scale.

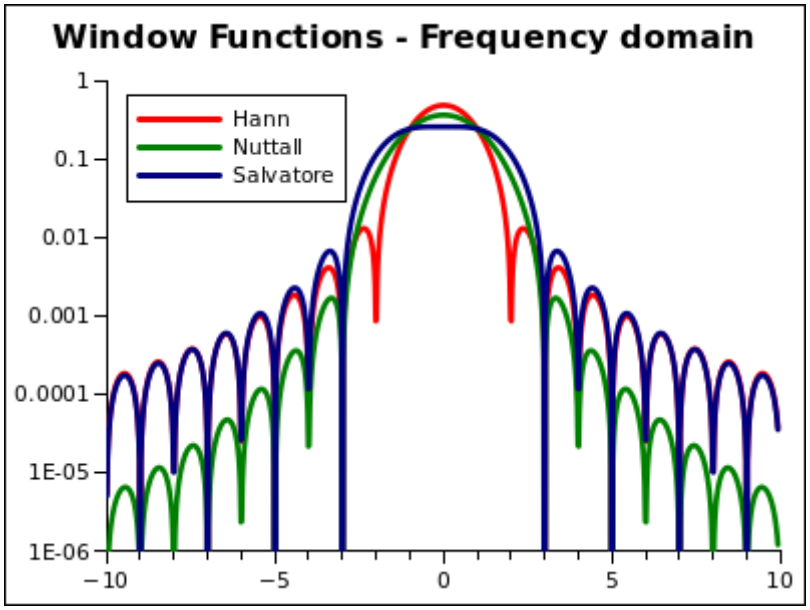
The Discrete Fourier Transform (DFT) is not nearly as fast as the Fast Fourier Transform (FFT) to find all the coefficients from 1 … $\frac{N}{2}$. But the DFT can be used to find a single coefficient and the FFT always computes N coefficients. Many FFT implementations force N to be a power of 2 or a product of small prime numbers. The DFT places no restrictions on N. With suitable choices of M and N, the DFT permits any set of frequencies to be measured up to the Nyquist limit $\frac{R}{2}$.

Window Functions



Fourier analysis assumes that the signal to be analyzed is a perfectly periodic function, with a period equal to the transform size. Real signals to be analyzed never fulfill this assumption. To reduce the error introduced, it is necessary to taper off the beginning and end of the samples to be analyzed by multiplying them by a window function. Window functions are defined over $0 \ldots 2\pi$ and equal zero at their endpoints with a maximum at the center. Many window functions have been defined, but their shapes are much alike.

The choice of window function has a definite impact on the results of the analysis. There is a tradeoff between removing noise and accuracy of amplitude measurements. This can be seen in a plot of some window functions in the frequency domain, computed according to the method in [Heinzel, Rudiger, & Schilling, 2002].

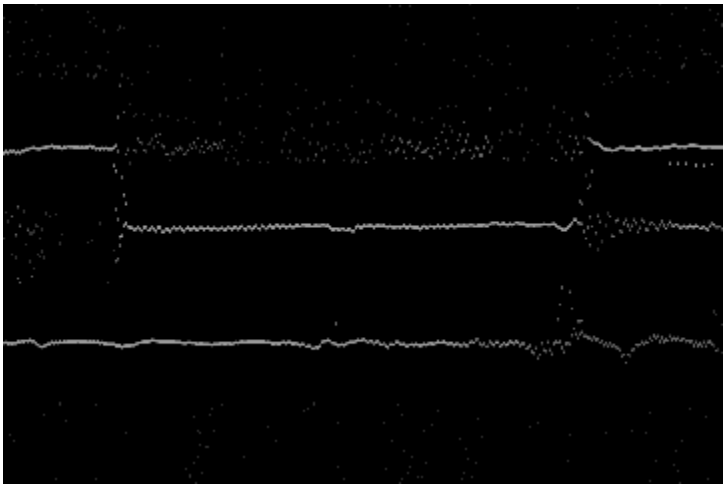


Visually, the difference between the functions may be described as a choice between flatness and sharpness of the center lobe. The Hann window has a fairly narrow center lobe, and it can better detect frequency. The Salvatore window has a flat center lobe, and it measures amplitude with greater accuracy.

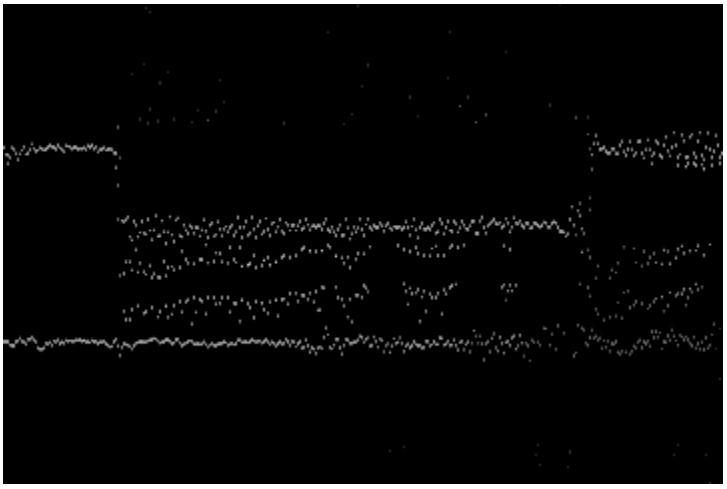
The Nuttall window has a flatness of < 1dB attenuation over the center bin, which means that it is flat enough for audio applications, since this is around the just-noticeable difference of human hearing. To not overburden the user with options, the Nuttall window has been selected for the project and is incorporated into subroutine FRAT. A different window function can be chosen by editing the parameter values in the source code.

The practical effects of window functions can perhaps be shown by example.

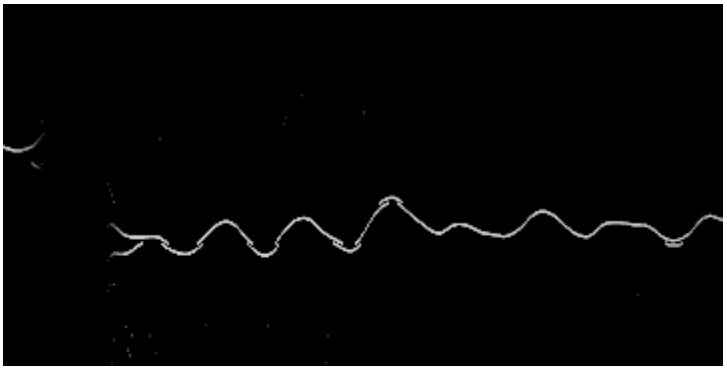
A very sharp window function like the Hamming has good resolution:



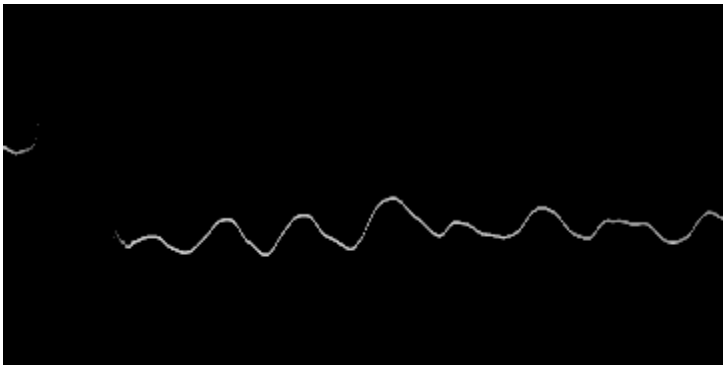
while a flattop window can be rather cloudy:



However, the comparison is not all in favor of the sharp windows. They seem to generate a forking artifact, where a single pitch splits (falsely?) into multiple frequencies:



The flattop window suffers much less from this effect:



Frequency Reassignment

The frequency resolution obtained by the Fourier transform is the best possible in the general case. However, often there is only one frequency component present in an interval of the spectrum. In this case, the time-frequency tradeoff may be dodged and much better frequency resolution can be obtained.

The idea is that since the sine and cosine coefficients of a component are known, the phase of the resultant wave is known. Frequency is the time derivative of phase. The derivation of the formula [Lazzarini, 2011] requires complex analysis which I don't pretend to understand, but the results are easy to use. Let u_j denote the window function, w_j denote the derivative of the window function with respect to index j, and y_j denote the input signal. To find the energy e and frequency $\hat{f}$, compute

$$
\begin{aligned}
a &= \frac{1}{N}\sum_{j=1}^{N} u_j y_j \cos\left(\frac{2\pi}{N} Mj\right) \\
b &= \frac{1}{N}\sum_{j=1}^{N} u_j y_j \sin\left(\frac{2\pi}{N} Mj\right) \\
c &= \frac{1}{N}\sum_{j=1}^{N} w_j y_j \cos\left(\frac{2\pi}{N} Mj\right) \\
d &= \frac{1}{N}\sum_{j=1}^{N} w_j y_j \sin\left(\frac{2\pi}{N} Mj\right) \\
e &= a^2 + b^2 \\
f &= \frac{MR}{N} \\
\hat{f} &= f + \left(\frac{ad - bc}{e}\right)\left(\frac{R}{2\pi}\right)
\end{aligned}
$$

Amplitude is the square root of energy.

Other Considerations

For best results, the input signal should be passed through a high-pass filter to remove frequencies below the fundamental. For efficiency, the signal is downsampled to keep N ≤ 6000. Before downsampling, the signal should be low-pass filtered to half the reduced sample rate. Digital filters have been discussed in my earlier article Customizable Butterworth Digital Filter.

Because there is a slight overlap of the frequency bins, there is a possibility that frequencies may be out of order. Insertion sort is efficient on almost sorted data. Subroutine `ISORTR` passively sorts on frequency. Subroutine `RORDER` by Robert Renka is then called to apply the ordering to the amplitudes and frequencies at each analysis frame.

Interpolation

The frequencies and amplitudes returned by `FRAT` must be used to set pixel intensities. That requires interpolation. There are several ways to do that, but Lanczos seems to be regarded as the best. That equation is:

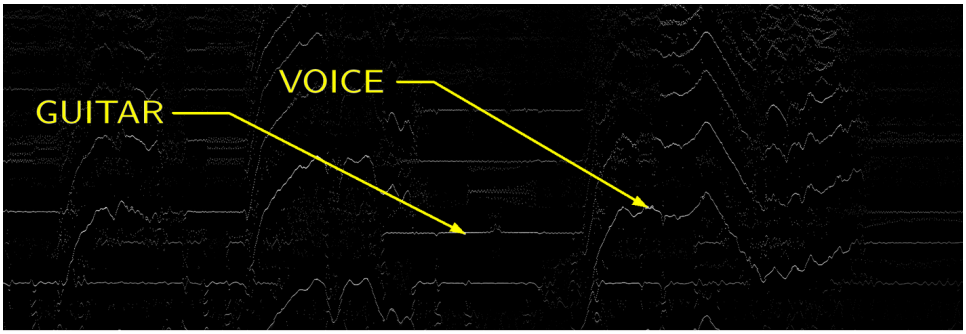
$$L(x) = \begin{cases} 1, & x=0 \\ \dfrac{\sigma \sin(\pi x) \sin\left(\frac{\pi x}{\sigma}\right)}{\pi^2 x^2}, & -\sigma < x < +\sigma,\ x \neq 0 \\ 0, & \text{otherwise} \end{cases}$$

L is a factor that tells how much to scale down the amplitude according to the distance x between the pixel position and the estimated frequency (both measured in pixels.) Parameter σ is 2 or 3. It is chosen σ = 2 in `SPECTR` to avoid the halo effect that occurs for σ = 3.

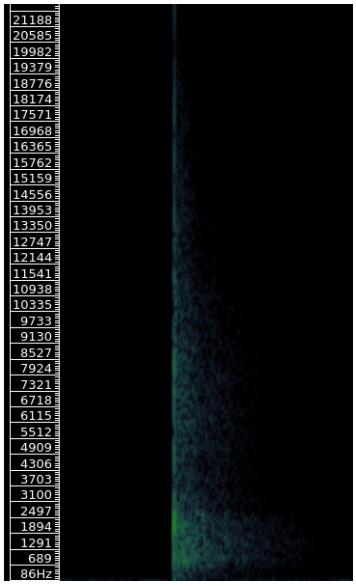
Finally, the logarithm of amplitude is taken, and the result is scaled into the range 0 to 255, the range of pixel intensities in a PGM file.

Interlude: Editing the Graphics

Learning to read a spectrogram takes some practice. Horizontal lines are pure tones at a single frequency:



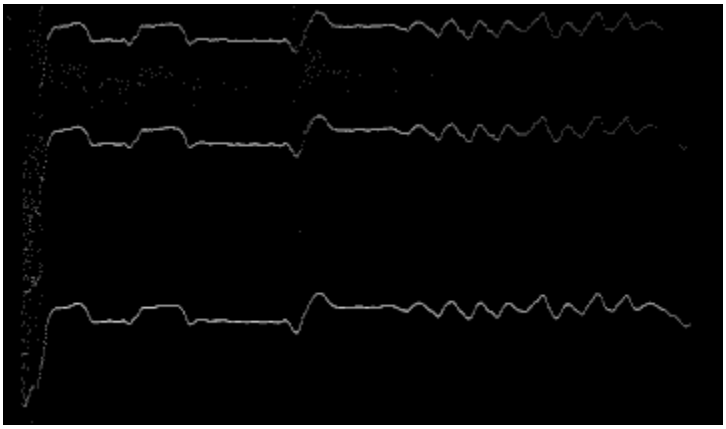
A stringed instrument like guitar produces a steady frequency. The human voice is not so steady. The vocal line is not nearly as straight, as seen in the above clip of "Delia's Gone" by Johnny Cash. Vertical lines are impulses. They sound like a click or a pop. *An impulse is activity at every frequency at once.* Here is a spectrogram of a handclap made with Sonic Visualiser:



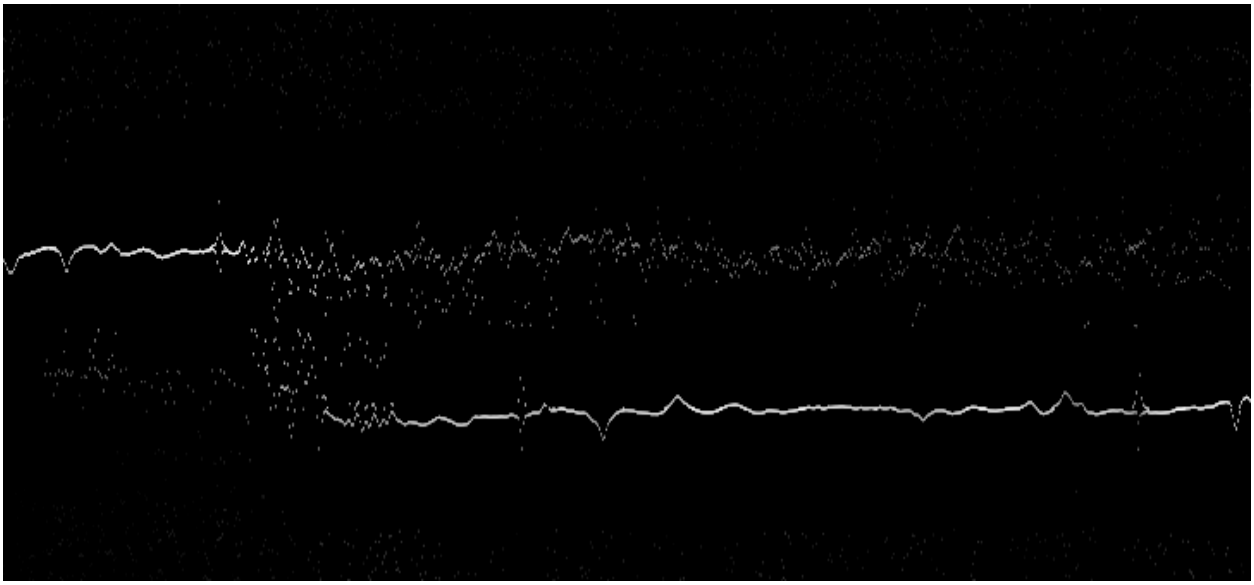
Cloudy regions are static noise:



Many musical instruments produce overtones at frequencies above the fundamental. These appear as parallel lines:



Sometimes there is an echo. This will show as a fainter line to the right of the original sound:



Identify the melody line, and erase the overtones, echoes, static, and clicks. It doesn't require a delicate touch. I recommend the black rectangle tool available in many editing programs. Fill in everything but the melody with solid black. If you miss a spot, program `SCRIBE` can handle a few stray pixels.

Part 2: From Melody Line to MIDI

Undoing Lanczos Interpolation

After the edited *.PGM* file is read, the pixel positions and intensities must be converted back to frequencies and amplitudes. In the special case that there are only two active pixels at a time point, it is possible to do this precisely. Let (x_o,y_o) denote the original, unknown frequency and

amplitude. Let (x_1,y_1) and (x_2,y_2) denote the pixel indices and intensities in the image, measured in linear light. Applying the Lanczos interpolation formula at each point gives:

$$y_1 = \frac{\sigma \sin(\pi(x_1 - x_0)) \sin(\frac{\pi}{\sigma}(x_1 - x_0))}{\pi^2 (x_1 - x_0)^2} y_0$$

$$y_2 = \frac{\sigma \sin(\pi(x_2 - x_0)) \sin(\frac{\pi}{\sigma}(x_2 - x_0))}{\pi^2 (x_2 - x_0)^2} y_0$$

Solve for y_o

$$y_0 = \frac{\pi^2 (x_1 - x_0)^2}{\sigma \sin(\pi(x_1 - x_0)) \sin(\frac{\pi}{\sigma}(x_1 - x_0))} y_1 = \frac{\pi^2 (x_2 - x_0)^2}{\sigma \sin(\pi(x_2 - x_0)) \sin(\frac{\pi}{\sigma}(x_2 - x_0))} y_2$$

and so the problem has been reduced to solving an equation for a single unknown:

$$\frac{(x_1 - x_0)^2}{\sin(\pi(x_1 - x_0)) \sin(\frac{\pi}{\sigma}(x_1 - x_0))} y_1 - \frac{(x_2 - x_0)^2}{\sin(\pi(x_2 - x_0)) \sin(\frac{\pi}{\sigma}(x_2 - x_0))} y_2 = 0$$

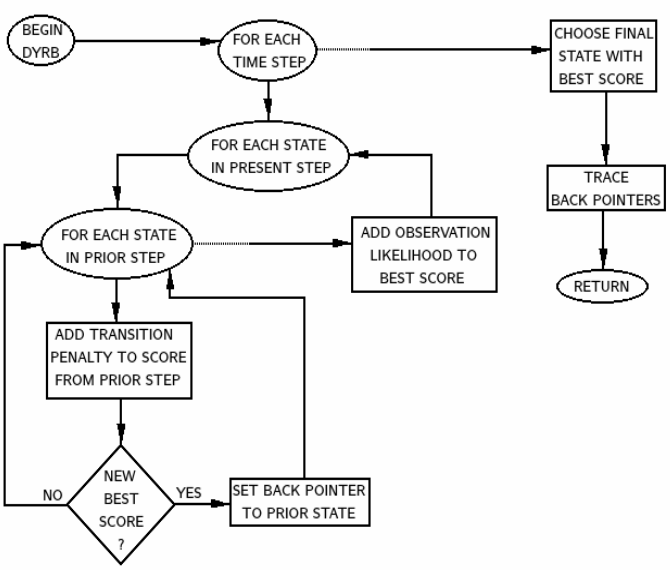
Subroutine `solv` is based on ZEROIN by Shampine and Watts. It is called at this point to solve numerically for x_o. Now substitute into the expression for y_o

$$y_0 = \frac{\pi^2 (x_1 - x_0)^2}{\sigma \sin(\pi(x_1 - x_0)) \sin(\frac{\pi}{\sigma}(x_1 - x_0))}$$

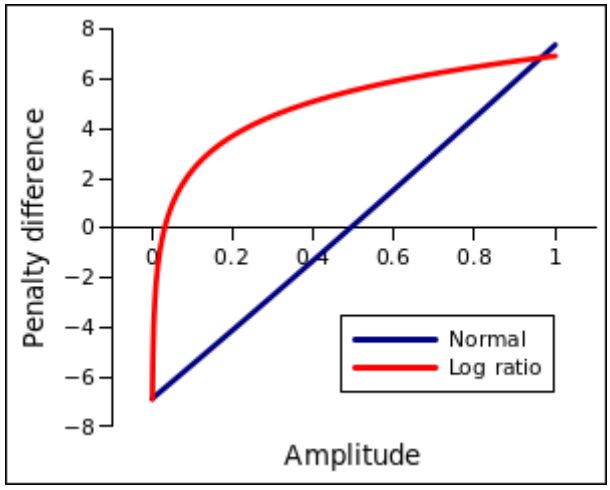
If more than two pixels are active in a frame, subroutine `QWM` is called for a fallback. It estimates frequency as a weighted average, and chooses amplitude as the maximum intensity.

Separating Sound from Silence

Next, it must be decided for each frame if a note is sounding or not. Setting a fixed threshold will not give good results. Tiny bright spots or dim spots will turn into many brief, unpleasant notes. A transition between sound and silence should only occur if it will last for a reasonable duration. The method of dynamic programming provides an optimal solution to these kinds of problems. [Temperley, 2001]



The Viterbi algorithm is the simplest form of dynamic programming. It asks at each step in the sequence, for each possible event, what event was most likely at the prior step? The answers for each step are kept, and at the end of the sequence, the most likely chain of events may be traced back. Subroutine DYRB implements the Viterbi algorithm for the special case of binary states (sounding or silent) and a real-valued sequence of observations (intensity).



I don't know the correct statistical model for inferring the probability of a note as a function of pixel intensity. The normal distribution does not seem appropriate for this application. Under the normal model, taking zero intensity and full intensity as the extremes, half intensity is indifference between sound or silence. This is not what we want; the indifference value should be much closer to zero. Instead, the penalty function chosen is the logarithm of intensity divided by a reference value, added or subtracted for sound or for silence.

The transition penalty is chosen by taking a small arbitrary unit of time. The value 0.035 seconds is chosen to match that in program MELISMA [Sleator & Temperley, 2000], a unit they dub a

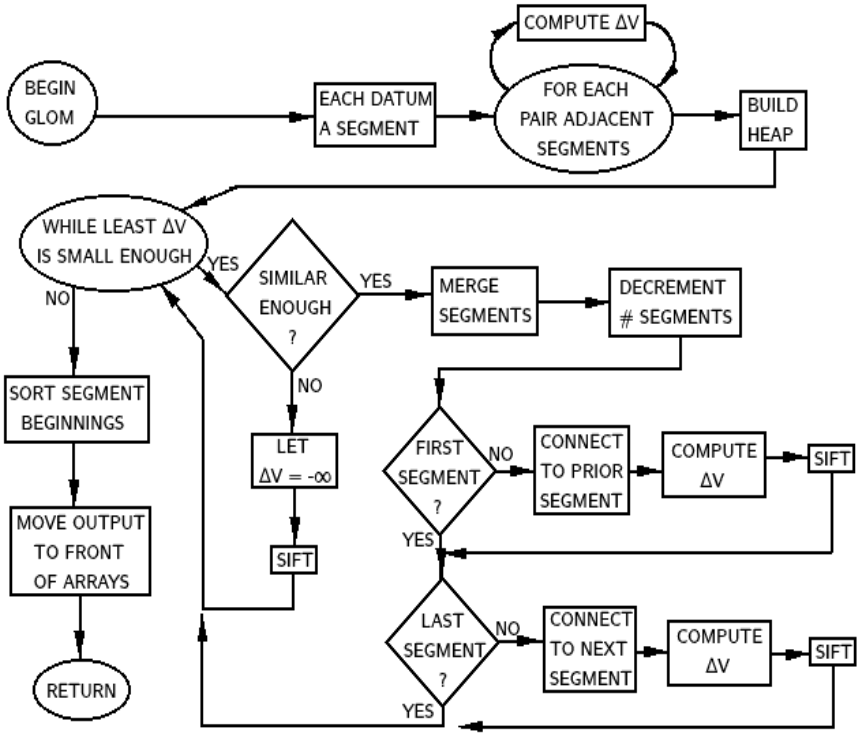
"pip." The transition penalty is a equal to a duration of a pip of the maximum observation penalty.

Thus the claim of optimality for dynamic programming must be hedged with a strong disclaimer. It is "optimal" given the assumed observation penalty and transition penalty.

The LULU Smoother

Next, outlier data is trimmed with a nonlinear smoother. Moving-averages have traditionally been used for this purpose. However, they are low-pass filters. As was shown above, an impulse contains all frequencies. Therefore, moving-average smoothers blur abrupt changes. Nonlinear smoothers can remove outlier points without blurring. [Jankowitz, 2007] Subroutine `SMOOTH` implements the Jankowitz "B" filter. It is based entirely on minimum and maximum operations, and does no arithmetic.

Agglomerating the Notes



Take the simple, greedy approach of merging the notes that will cause the least increase in sum of squared difference between frequency of each pixel and the segment average. This is not an optimal algorithm, but it is efficient and has been shown to work well in practice. [Terzi & Tsaparas, 2006] As each segment grows, its average frequency changes, and thus the potential change in error from merging it with its neighboring segments also changes. Since agglomeration is a 1-dimensional case of data clustering, Ward's formula applies. [Theodoridis & Koutroumbas, 2009]

$$e_{ij} = \frac{n_i\, n_j}{n_i + n_j} d_{ij}$$

where *e* is the error, *d* is the difference between segment averages, and *n* is the number of objects in each segment. These updates can be handled efficiently by putting the potential changes in error into a heap and sifting the least change to the top.

Subroutine `GLOM` takes as input a real-valued sequence and three parameters. `ADPARM` is the threshold difference between the average values of adjacent segments. `MDPARM` is the threshold value of range between the maximum and minimum values within a segment. `K` is a desired number of segments. Segments are merged until a stopping criterion is met. Either the difference between every pair of adjacent segments exceeds `ADPARM`, or no segments can be merged without exceed `MDPARM`, or the number of segments `K` has been reached.

The Expression Formula

In program `SPECTR`, the logarithm of the Fourier coefficients was taken and the result was scaled to the range of pixel intensities. According to the MIDI standard, note velocity is to have an exponential interpretation. Thus, the velocity may be taken directly proportional to maximum pixel intensity in a note, $V = \frac{127}{255} Y$, where *Y* is intensity from 0 to 255 and *V* is velocity from 0 to 127.

The situation for MIDI expression is more difficult. Expression is a maximum by default, and is set to create diminuendos within each note. See the source code for a derivation of the result:

$$E = 127 \sqrt{\epsilon}^{\frac{V}{127} - \frac{Y}{255}}$$

where $\epsilon = \frac{1}{1024}$, the resolution parameter in program `SPECTR`.

Topics Not Covered

Reading and writing the *.WAV, .PGM,* and *.MID* file formats is not very interesting. For information on them, please see the references.

Build Instructions

To compile the C code with `gcc`, do:

```
gcc -o spectr -O2 rorder.c safdiv.c isortr.c search.c filter.c frat.c spectr.c
-lm
gcc -o scribe -O2 solv.c ishell.c rorder.c safdiv.c isortr.c search.c sift.c
glom.c dyrb.c smooth.c scribe.c -lm
```

To compile the FORTRAN with `gfortran`, do:

```
gfortran -o spectr -O2 -fno-automatic wrapper.f both.f rorder.f safdiv.f
isortr.f search.f filter.f frat.f spectr.f
gfortran -o scribe -O2 -fno-automatic wrapper.f both.f zerora.f ishell.f
safdiv.f search.f sift.f glom.f dyrb.f smooth.f scribe.f
```

That's a bit to type, but there are no dependencies, no makefiles, and no header files.

Using the Code

Usage is:

```
spectr something.wav something.pgm
scribe something.pgm something.mid
```

where:

- *.wav* is standard Microsoft wave audio
- *.pgm* is Portable Grey Map image file
- *.mid* is your output

You are free to attempt any kind of transform on the image except that it has to remain 640 pixels in height. Remember that `SCRIBE` transcribes solos only.

Both programs have an option `--tune` to transpose. So you can do something like:

```
spectr --tune +24.5 deep.wav alto.pgm
scribe --tune -24.5 alto.pgm deep.mid
```

That shifts a baritone voice up by two octaves and a quarter-tone into the range of the spectrogram, then transposes it back when writing the MIDI.

Program `SCRIBE` has the options `--patch`, `--text`, `--copyright`, and `--seqname` so that

```
scribe --patch 68 --seqname "Hacker's Blues" --copyright "(C)2021 Joe"
       --text "nobody reads the comments" blues.pgm blues.mid
```

sets the patch to oboe, gives the track a name, a copyright notice, and a comment.

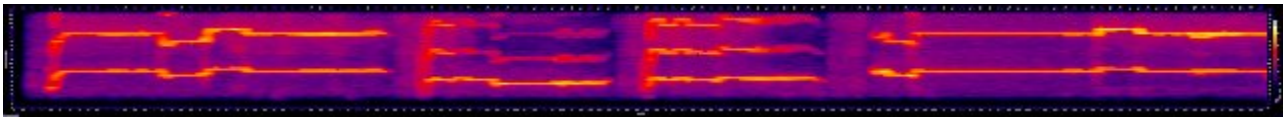
Program `SPECTR` has an option `--swab` which you might need if your *.WAV* is a 32-bit float with byte order backwards from the computer's default.

Example Problems

Vocal

Consider this sample of singing by Katy, courtesy of digifishmusic. [Katy, 2007]

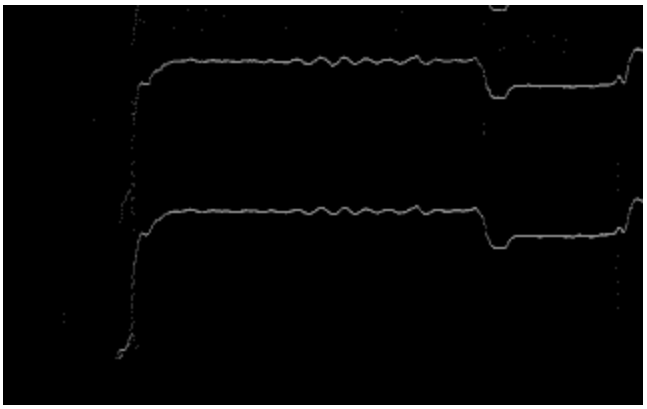
An ordinary spectrogram (made with sox) is fairly blurry.



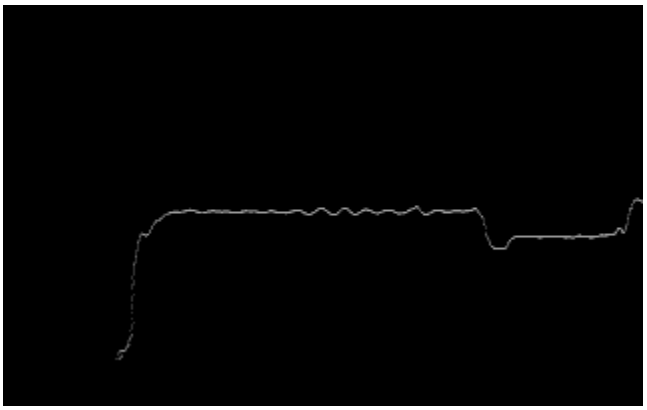
A high-resolution spectrogram made with SPECTR shows the details of the tremolo.



Next, edit the image with Windows paint or any similar program to remove the overtones.



Erase all but one of the parallel lines:



Because this is a standard image file, *any* editing operation which can be done on an image may be done at this step. Now process the image with SCRIBE to produce the MIDI file.

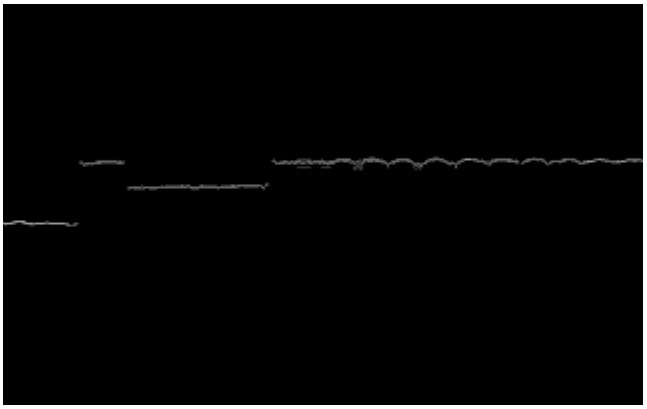
For comparison, the result from the free file converter at ofoct.com is available. This service is free and does not require you to install anything on your computer. Both MIDIs accurately capture the performance. The competitor's MIDI is very concise. On the other hand, it contains 167 notes. The output of `SCRIBE` is made long by many changes in expression, but there are only 28 notes. In this sense, it is a more accurate transcription.

Instrumental

A flute performance [kerri, 2007] was processed by the SCAT system. A detail of the spectrogram:



After editing the image:



The result is a reasonably good transcription, although some of the ornaments have been lost.

Release History

- 3 12th August, 2023: Agglomeration seeks least squared error.
- 2 30th April, 2022: Nonlinear smoothing, inverse interpolation, no *.exe* files
- 1 10th October, 2021: Add readme, FORTRAN source and *.exe* files
- 0 6th October, 2021: Original release to the MidKar group

References

1. "PGM Format Specification" by Jef Poskanzer, 1989. online
2. "Calculation of a constant Q spectral transform," Judith C. Brown, Journal of the Acoustic Society of America v.89 i.1, January 1991. Available at researchgate.
3. "Standard MIDI Files Specification", MIDI Manufacturers Association, 1996. online
4. "Microsoft WAVE soundfile format", Craig Stuart Sapp, 1997. online
5. *The Cognition of Basic Musical Structures*, David Temperley, MIT Press, 2001. First chapter available from the publisher.
6. "Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new flat-top windows," G. Heinzel, A. Rudiger and R. Schilling, Max-Planck-Institut fur Gravitationsphysik, February 15, 2002. Available at researchgate.
7. "Cookbook formulae for audio EQ biquad filter coefficients," Robert Bristow-Johnson, [2005?]. Available at musicsdp.
8. "The Butterworth Low-Pass Filter," John Stensby, 19 Oct 2005. Available from the University of Alabama.
9. "Efficient Algorithms for Sequence Segmentation," Evimaria Terzi and Panayiotis Tsaparas, *Proceedings of the Sixth SIAM International Conference on Data Mining*, April 2006. Available at researchgate.
10. *Some Statistical Aspects of LULU Smoothers*, Maria Dorothea Jankowitz, Dissertation, Stellenbosch University, Dec. 2007. Available from Stellenbosch University.
11. "Haunted Canyon Flute," by kerri, 2007. At freesound.
12. "Katy Sings LaaOooAaa," by Katy, recorded by digifishmusic, 2007. At freesound.
13. *Pattern Recognition* Sergios Theodoridis and Konstantinos Koutroumbas, Academic Press / Elsevier, 2009.
14. Bear File Converter, 2009.
15. "Programming the Phase Vocoder" by Victor Lazzarini, in *The Audio Programming Book*, ed. Richard Boulanger and Victor Lazzarini. MIT Press, 2011.

License

This article, along with any associated source code and files, is licensed under A Public Domain dedication